The required packages are:

- express
- express-validator
- i18n

==========

To configure the modules with the Express module:

```
// Configure i18n
i18n.configure({
  locales: ['en', 'es', 'fr'],
  directory: path.join(__dirname, 'locales'),
  defaultLocale: 'en',
  queryParameter: 'lang',
  autoReload: true,
  updateFiles: false,
  objectNotation: true,
  register: global
})

// Middleware
app.use(express.json())
app.use(express.urlencoded({ extended: true }))
app.use(i18n.init)
```

==========

In the validator files, we should be careful when handling the error messages:

- Ex-01: For the login requests:

```
const { body } =require('express-validator')

// Login Validation
const loginValidation = [
  body('email')
    .notEmpty()
    .withMessage('validation.required')
    .isEmail()
    .withMessage('validation.email'),

  body('password')
    .notEmpty()
    .withMessage('validation.required')
];

module.exports = loginValidation
```

- Ex-02: For the register requests:

```
const { body } =require('express-validator')

// User Registration Validation
const userRegistrationValidation = [
  body('username')
    .trim()
    .notEmpty()
    .withMessage('validation.required')
    .isLength({ min: 3, max: 20 })
    .withMessage('validation.minLength')
    .custom((value) => {
      // Simulate database check
      const takenUsernames = ['admin', 'user', 'test'];
      if (takenUsernames.includes(value.toLowerCase())) {
        throw new Error('validation.custom.usernameTaken');
      }
      return true;
    }),

  body('email')
    .trim()
    .notEmpty()
```

```
  .withMessage('validation.required')
  .isEmail()
  .withMessage('validation.email')
  .normalizeEmail(),

body('password')
  .notEmpty()
  .withMessage('validation.required')
  .isLength({ min: 8 })
  // .withMessage({key: 'validation.minLength', args: { min: 3 }})
  .withMessage('validation.minLength')
  .custom((value) => {
    const hasUpperCase = /[A-Z]/.test(value);
    const hasLowerCase = /[a-z]/.test(value);
    const hasNumbers = /\d/.test(value);

    if (!hasUpperCase || !hasLowerCase || !hasNumbers) {
      throw new Error('validation.custom.weakPassword');
    }
    return true;
  }),

body('confirmPassword')
  .notEmpty()
  .withMessage('validation.required')
  .custom((value, { req }) => {
    if (value !== req.body.password) {
      throw new Error('validation.passwordMatch');
    }
    return true;
  }),

body('age')
  .optional()
  .isInt({ min: 13, max: 120 })
  .withMessage('validation.numeric'),

body('website')
  .optional()
  .isURL()
  .withMessage('validation.url'),

body('role')
  .optional()
  .isIn(['user', 'admin', 'moderator'])
```

```
    .withMessage('validation.oneOf')
];

module.exports = userRegistrationValidation
```

==========

To build the validation middleware:

```
const { validationResult } = require('express-validator')

class ValidationMiddleware {
  // Format errors with i18n support
  static formatErrors(req, errors) {
    return errors.array().map(error => {
      let message = error.msg;

      // If message is a translation key, translate it
      if (message.startsWith('validation.')) {
        message = req.__(message);

        // Handle interpolation for min/max values
        if (error.msg === 'validation.minLength' && error.options) {
          message = message.replace('{{min}}', error.options.min);
        }
        if (error.msg === 'validation.maxLength' && error.options) {
          message = message.replace('{{max}}', error.options.max);
        }
        if (error.msg === 'validation.oneOf' && error.options) {
          message = message.replace('{{values}}', error.options.join(', '));
        }
      }

      return {
        field: error.path,
        message: message,
        value: error.value,
        location: error.location
      };
    });
  }

  // Main validation handler
  static handle(validations) {
    return async (req, res, next) => {
      // Execute all validations
      await Promise.all(validations.map(validation => validation.run(req)));
```

```
      const errors = validationResult(req);

      if (!errors.isEmpty()) {
        const formattedErrors = this.formatErrors(req, errors);

        return res.status(422).json({
          success: false,
          message: req.__('Validation failed'),
          errors: formattedErrors,
          // Include locale info for debugging
          locale: req.getLocale()
        });
      }

      next();
    };
  }
}

module.exports = ValidationMiddleware;
```

==========

Create the register router:

```
const router = require('express').Router()

// Registration route
router.post('/api/register',
  (req, res) => {
    res.json({
      success: true,
      message: req.__('Registration successful'),
      data: {
        username: req.body.username,
        email: req.body.email
      }
    });
  }
);

module.exports = router
```

==========

Create the login router:

```
const router = require('express').Router()

// Login route
router.post('/api/login',
  (req, res) => {
    res.json({
      success: true,
      message: req.__('Login successful'),
      token: 'jwt-token-here'
    });
  }
);

module.exports = router;
```

==========

Register the login and register routers with the middleware:

```
app.use('/', ValidationMiddleware.handle(userRegisterValidation), registerRouter)
app.use('/', ValidationMiddleware.handle(loginValidation), loginRouter)
app.use('/', localeRouter)

// Error handler
app.use((err, req, res, next) => {
  console.error(err.stack)
  res.status(500).json({
    success: false,
    message: req.__('Internal server error')
  })
})
```

==========